

Ingeniería del Software II

Práctica #1 – Mutation Analysis

Ejercicio 1

Sea el siguiente programa:

```
def foo(A: list[int], B: list[int]) -> int:
    r:int = 0
    i:int = 0
    while i < len(A):
        r += A[i] * B[i]
        i += 1
    return r
```

Escribir una expresión booleana que sea la pre-condición más débil que impida que se produzcan excepciones `TypeError` (por acceso a null) e `IndexError` (por acceso fuera de rango en un arreglo) durante la ejecución del programa.

Ejercicio 2

Sea el siguiente programa:

```
def foo(x:int, y:int) -> int:
    z:int = 0
    if x <= y:
        z = x
    else:
        z = y
    return z
```

Y el siguiente test suite:

```
class TestFoo(unittest.TestCase):
    def test1(self):
        rv = foo(1, 0)
        self.assertEqual(rv, 0)
```

Proveer un nuevo test case que logre 100% de cobertura de sentencias y de branches

Ejercicio 3

Sea el siguiente programa python:

```
def foo(x:int, y:int) -> int:
    z:int = 0
    if x <= y:
        z = x
    else:
        z = y
    return z
```

Y el siguiente mutante al que llamaremos mutante #1:

```
def foo(x:int, y:int) -> int:
    z:int = 0
    if x != y: #mutacion
```

```

        z = x
    else:
        z = y
    return z

```

Y el siguiente test suite:

```

class TestFoo(unittest.TestCase):
    def test1(self):
        self.assertEqual(foo(0, 1), 0)

    def test2(self):
        self.assertEqual(foo(0, 0), 0)

```

Extender el test suite con un nuevo test capaz de detectar al mutante #1.

Ejercicio 4

Sea el siguiente programa que clasifica un triángulo por sus lados:

```

def triangle(a:int, b:int, c:int) -> int:
    if a <= 0 or b <= 0 or c <= 0:
        return 4 # invalid
    if not (a + b > c and a + c > b and b + c > a):
        return 4 # invalid
    if a == b and b == c:
        return 1 # equilateral
    if a == b or b == c or a == c:
        return 2 # isosceles
    return 3 # scalene

```

Escribir un mutante no-equivalente del mismo y presentar un test case capaz de matar al mutante.

Ejercicio 5

Sea el siguiente programa.

```

def test_me(n: int) -> int:
1:  r:int = 0
2:  if n >= 0:
3:      i:int = 0;
4:      while i < n:
5:          if i % 2 == 1:
6:              r = r+i
7:              i = i+1
8:  return r

```

Usando el operador de mutación ROR¹, exhibir mutantes tales que:

- Un mutante con un test que lo detecta (indicando la línea que cambió).
- Un mutante equivalente tal que no exista test que lo detecte (indicando la línea que cambió).

En caso de no existir alguno de estos mutantes, explicar por qué.

Ejercicio 6

Sea el siguiente programa.

¹modifica una comparación aritmética reemplazandola por <, >, <=, >=, ==, !=, false, true

```
def test_me(x: int, y: int) -> bool:
    result: bool = False
    z: int = 2*y
    if z == x:
        if x > y+10:
            result = True
    return result
```

Usando el operador de mutación AOR²:

- Exhibir un mutante con un test que lo detecta (indicando la línea que cambió).
- Exhibir un mutante equivalente tal que no exista ningún test que lo detecte (indicando la línea que cambió).

En caso de no existir alguno de estos mutantes, explicar por qué.

Ejercicio 7

Sea el siguiente programa:

```
1 def test_me(k: int, j: int) -> int:
2     i: int = 0
3     r: int = 0
4     while i < 2: # C1
5         if k == j: # C2
6             r = r+1
7             i = i+1
8     return r
```

Exhibir usando el operador de mutación ABS³:

- Un mutante con un test que lo detecte. Indicar cuál es el test case que lo detecta.
- Un mutante equivalente tal que no exista ningún test case que lo detecte.

En ambos casos indicar la línea que se modificó.

Ejercicio 8

Sea el siguiente programa:

```
1 def test_me(k: int, j: int) -> int:
2     r: int = 1
3     if k != 0 and k > 0: #C1
4         c: int = k % 3 # k modulo 3
5         i: int = 0
6         while i < c: #C2
7             r = r*j
8             i = i+1
9     return r
```

Exhibir usando el operador de mutación ROR⁴:

- Un mutante con un test que lo detecte. Indicar cuál es el test case que lo detecta.
- Un mutante equivalente tal que no exista ningún test case que lo detecte.

²modifica una expresión aritmética binaria reemplazándola por +, -, *, /, %, **, x, y

³modifica una expresión aritmética E reemplazándola por $abs(E)$, $-abs(E)$, o la constante 0

⁴modifica una condición aritmética binaria reemplazándola por >, <, ≥, ≤, ==, !=

En ambos casos indicar la línea que se modificó.

Ejercicio 9

Sea el siguiente programa:

```
1 def test_me(k: int, j: int) -> int:
2   r: int = 1
3   if k>0 and k<3: #C1
4     i: int = 0
5     while i<k: #C2
6       r = r*j
7       i = i+1
8   return r
```

Exhibir usando el operador de mutación ABS⁵:

- Un mutante con un test que lo detecte. Indicar cuál es el test case que lo detecta.
- Un mutante equivalente tal que no exista ningún test case que lo detecte.

En ambos casos indicar la línea que se modificó.

⁵modifica una expresión aritmética E reemplazándola por $abs(E)$, $-abs(E)$, o la constante 0